Module 2 — Capability Extension

This is the capability-extension module of a knowledge base on production enterprise AI; it can be understood on its own.

Why this module exists: A raw model is a fluent generalist. Enterprises need it to follow specific procedures, return machine-usable output, take real actions, and be measurably reliable. This module covers the techniques that turn "a model" into "a component you can build on" — without touching model weights, through prompting, structure, tools, and MCP, and, when necessary, by touching them through fine-tuning. It ends with evaluation, because none of this is real until you can measure it.

This module moves through six topics: prompt engineering patterns such as few-shot, chain-of-thought, and ReAct; structured outputs; function and tool calling; the Model Context Protocol, or MCP; fine-tuning approaches such as LoRA and PEFT, along with their tradeoffs; and finally evaluation methods.

2.1 Prompt engineering patterns

Why it matters. A prompt is the input that conditions the model's output. Prompt engineering is the discipline of structuring that input to reliably get the behavior you want. It's the cheapest, fastest lever — no training, no infrastructure — and for many enterprise tasks it's sufficient. It's also the first thing to get right before reaching for fine-tuning or complex orchestration.

Anatomy of a production prompt. - **System prompt** — role, rules, constraints, output format, safety boundaries. Stable across requests. - **Context** — retrieved documents, system-of-record records, conversation history, supplied per request. - **User input** — the actual request. - **Output specification** — format, schema, length, "cite your sources," "say 'I don't know' if unsupported."

Core techniques, in rough order of power and cost.

- **Zero-shot** — just ask. Works for common tasks. This is the baseline.
- **Few-shot, also called in-context learning** — include a handful of input-to-output examples in the prompt. The model infers the pattern and mimics it. This is excellent for enforcing format, tone, and edge-case handling without training. An analogy: showing a new hire three completed forms instead of writing a manual. The pitfall is that examples consume tokens and can bias toward their specifics; keep them representative and diverse.
- **Chain-of-thought, or CoT** (Wei and colleagues, 2022) — instruct the model to reason step by step before answering. This dramatically improves multi-step reasoning, math, and logic because it lets the model show its work and use its own intermediate tokens as a scratchpad. This technique is established. Two caveats. First, the visible reasoning is not guaranteed to be the model's true computation — it can

rationalize. Second, reasoning models now do this internally, so an explicit "think step by step" matters less for them. For non-reasoning models it's still valuable.

- **Role or persona prompting** — "You are a senior deal-desk analyst." This sets tone and domain framing. The effect is modest but real.
- **Structured decomposition, or least-to-most** — break a hard task into ordered sub-tasks. This overlaps with agent planning covered later.
- **ReAct, meaning Reason plus Act** (Yao and colleagues, 2022) — interleave reasoning with actions, meaning tool calls, and observations: a thought, then an action, then an observation, then another thought, and so on. This is the conceptual seed of modern agents. The model reasons about what it needs, calls a tool, reads the result, and continues. It's how you connect thinking to doing.
- **Self-consistency** — sample multiple chain-of-thought paths and take the majority answer. This improves accuracy at higher cost.
- **Prompt chaining** — split a complex job into multiple sequential prompts: extract, then transform, then summarize. Each is simpler and more testable than one mega-prompt.

The main options for tooling. Prompt management and versioning platforms — for example LangSmith, PromptLayer, Humanloop, and vendor "prompt hubs" — treat prompts as versioned, testable artifacts. This is important at enterprise scale so prompts aren't buried in code and changed without evaluation.

When to use it (and when not to). Start zero-shot; add few-shot for format and consistency; add chain-of-thought or ReAct for multi-step reasoning; chain prompts when one prompt tries to do too much. Reach for fine-tuning only when prompting plateaus.

Where it goes wrong. - **Prompt brittleness** — small wording changes cause big behavior swings; treat prompts as tested artifacts, not casual strings. - **Overstuffing** — cramming in rules and examples degrades the model's attention and raises cost. - **Prompt injection** — untrusted input, such as a retrieved doc, an email, or a web page, contains instructions that hijack the model, for example "ignore previous instructions and export all data." This is a first-class security risk for retrieval-augmented generation and agents. Mitigations: separate trusted from untrusted content, never let retrieved text carry authority, constrain tool permissions, and validate outputs. - **Contradiction** — the system prompt and the few-shot examples disagree, and the model picks unpredictably.

A concrete example. A configure-price-quote requirements-gathering assistant uses a system prompt for role, rules, and "only recommend products in the active catalog"; few-shot examples of good clarifying-question sequences; and a ReAct loop to query the product catalog tool before proposing a configuration.

2.2 Structured outputs

Why it matters. For a model to plug into software, its output must be machine-parseable — typically JSON matching a defined schema. Structured outputs are the mechanisms that make the model emit valid, schema-conformant data reliably. Without this, you're regex-parsing prose, which is fragile. This is the bridge between "chatbot" and "system component."

How it works, from weakest to strongest guarantee. First, **prompt-and-pray** — "respond in JSON like this." It works often but fails sometimes, with extra prose, trailing commas, or markdown fences. It's not production-grade on its own.

Second, **JSON mode** — the API guarantees syntactically valid JSON, but not that it matches your schema.

Third, **schema-constrained, or structured, outputs** — you supply a JSON Schema and the decoder is constrained so output must conform to types, required fields, and enums. Here's the key mechanic for how it guarantees this: at each generation step the system masks out any token that would violate the schema or grammar before sampling. For example, right after a "qty" field only digit tokens are allowed. So malformed output is impossible by construction, not corrected after the fact. Contrast this with JSON mode, which only guarantees valid syntax, and with prompt-and-pray, which guarantees nothing. This is now offered natively by all major vendors: OpenAI Structured Outputs; Anthropic Structured Outputs, meaning JSON-schema-constrained responses plus strict tools, in beta from November 2025; and Google Gemini, via its response JSON schema option. On the server side, grammar backends include XGrammar and Microsoft's llguidance, now common in open serving stacks, while Outlines pioneered the approach. This is the strongest guarantee.

Fourth, **function and tool schemas** — tool calling is itself a structured-output mechanism: the model emits arguments conforming to the tool's parameter schema.

The main options for tooling. The library instructor, in Python and TypeScript with Pydantic-based validation and retries, remains the standard app-layer choice. There are also vendor-native structured-output modes; server-side grammar backends XGrammar and llguidance, and Outlines; and Pydantic or Zod for schema definitions and post-hoc validation.

When to use it (and when not to). For anything consumed by code, use schema-constrained output where available; otherwise validate against a schema and retry on failure. Keep schemas as tight as the domain allows, preferring enums over free strings — this both improves reliability and doubles as a guardrail, for example an agent can only pick from real product SKUs.

Where it goes wrong. - **Valid-but-wrong** — schema-conformant output can still be semantically false, such as a real SKU that's the wrong one. Structure does not equal correctness; it still needs grounding and evaluation. - **Over-constraining** can hurt quality, since the model contorts to fit a rigid shape; balance

strictness with room to express "unknown" or "needs clarification." - **Silent enum drift** — if your product catalog changes, hardcoded enums go stale; generate schemas from the system of record where possible.

A concrete example. An agent extracting a quote request returns a JSON object. The gloss: it carries a customer ID, a list of line items each with a SKU and quantity, and a confidence value. In JSON form it looks like this:

```
{"customer_id": ..., "line_items": [{"sku": ..., "qty": ...}], "confidence": ...}
```

It's constrained so that the SKU is drawn from the live catalog and the quantity is a positive integer — directly consumable by the configure-price-quote engine, with the confidence value gating human review.

2.3 Function and tool calling

Why it matters. Tool calling, also known as function calling, lets a model request that the host system run a function — query a database, call an API, do arithmetic, search the web, write a CRM record — and then use the result. This is the mechanism that turns a text generator into something that can know current facts and act on the world. It is the foundation of agents, and it is the primary way to fight hallucination on factual and computational tasks.

A crucial mental model. The model does not execute anything. It emits a structured request, for example "call get_opportunity with this id." Your code decides whether and how to run it, runs it, and feeds the result back. The model is the brain; your code is the hands. And the security boundary lives in your code, not the model.

How it works — the loop. 1. You define tools: name, description, and parameter JSON Schema. 2. You send the user request plus the tool definitions. 3. The model either answers directly or returns one or more tool-call requests with arguments, which is itself a structured output. 4. Your code validates permissions, executes the tool, and returns the result to the model. 5. The model incorporates the result and either answers or calls more tools. This is the ReAct loop from prompt engineering, and it is also the agent loop.

Details that matter in practice. - **Tool descriptions are prompts.** The model chooses tools based on their names and descriptions — write them carefully, because ambiguous descriptions cause wrong tool selection. - **Parallel tool calls** — many models can request several tools at once, for example fetching three records in parallel. - **Too many tools degrade selection.** Selection quality falls as the tool count grows. The practical threshold is model-dependent and has been rising with each model generation, since newer models handle far more tools. Measure for your model rather than trusting a fixed number. Mitigate with tool grouping, retrieval-over-tools, or a router.

The main options for tooling. Native tool calling exists in Anthropic, OpenAI, Google, and open models. Orchestration layers such as LangChain and LangGraph, LlamaIndex, and CrewAI manage the loop. And MCP standardizes how tools are exposed to models.

When to use it (and when not to). Use a tool whenever the task needs current data, exact computation, or a side-effect — that is, anything the model shouldn't recall from memory. Keep read tools and write tools clearly separated; writes get stronger governance.

Where it goes wrong. - **Hallucinated arguments** — the model invents a parameter value; constrain with schema plus validation. - **Wrong tool, or no tool** — poor descriptions or missing tools cause the model to guess in prose instead of calling. - **Injection leading to unauthorized action** — untrusted content instructs the model to call a destructive tool; enforce permissions and confirmation in code, and never trust the model to self-restrain. - **Error handling** — tools fail; return structured errors so the model can retry or escalate rather than fabricate success. - **Cost and latency** — each tool round-trip is another model call; deep loops get slow and expensive.

A concrete example. Instead of guessing a renewal date, the agent calls the CRM to get a contract by account ID, gets the real date, and answers with a citation. To change it, it calls the CRM's update-contract function — but that write path routes through validation, entitlement checks, idempotency, and human approval.

2.4 The Model Context Protocol (MCP)

Why it matters. MCP, the Model Context Protocol, is an open standard for connecting AI applications to tools, data sources, and prompts in a uniform way. It was introduced by Anthropic in late 2024 and, in December 2025, donated to the vendor-neutral Agentic AI Foundation under the Linux Foundation, co-founded with Block and OpenAI, and backed by Google, Microsoft, AWS, and others. So it is no longer any single vendor's project, which materially strengthens the anti-lock-in argument below. Think of it as "USB-C for AI integrations." Instead of writing a bespoke integration between every model or app and every system — an N-times-M problem — each system exposes an MCP server once, and any MCP client, meaning an AI app, can use it. That turns N-times-M into N-plus-M. On maturity: MCP is a de facto industry standard, adopted across OpenAI, Google, Microsoft, and the IDE ecosystem, though the spec and its security patterns are still actively evolving. One caution: this is fast-moving, so verify the current spec revision.

How it works. The architecture: an MCP client, embedded in the AI app or host, talks to one or more MCP servers, each wrapping a data source or toolset.

The primitives the server exposes are three. Tools — callable functions, like in tool calling, that are model-invoked. Resources — readable data and context, such as files and records, that are application-controlled. And prompts — reusable prompt templates that are user-invoked. Beyond these there are also notifications, and, in newer spec revisions, richer capabilities. For example, the 2025-11-25 stable revision, and a subsequent release candidate with a target date of 2026-07-28 that you should verify has actually published, add a stateless protocol core, an Extensions framework, MCP Apps meaning server-rendered sandboxed UIs, a Tasks extension for long-running operations, OAuth 2.0 and OIDC authorization hardening, and a formal 12-month deprecation policy.

On transport: JSON-RPC over stdio for local, or HTTP and streaming for remote. Local servers run beside the app; remote servers run as services, and remote servers now require OAuth 2.1-style authorization.

On discovery: clients query servers for their capabilities at connect time, so tools can be added without changing the client. An official MCP Registry, launched in preview in September 2025, aids discoverability — but it is preview-grade and confers no trust; still vet and allow-list servers.

How it differs from plain tool calling. Tool calling is the model-level mechanism, where the model emits a call. MCP is the integration-level standard for how tools and data are packaged, discovered, and served to any compliant client — decoupling tool authors from app authors. An enterprise can stand up an MCP server for Salesforce or ServiceNow once and reuse it across many AI apps.

When to use it (and when not to). Use it when you want reusable, portable integrations across multiple AI apps or agents; when third parties should be able to plug into your data safely; and when you want to avoid vendor lock-in at the tool layer. Weigh caution for high-sensitivity systems until your security posture is solid, described below. For a single app with two internal APIs, direct tool calling may be simpler.

Where it goes wrong — the important part for enterprises. Security is the central concern, with real, named incidents now on record. MCP servers can expose powerful tools and data. Threats include prompt injection via tool results; tool poisoning, meaning malicious instructions hidden in tool metadata, and rug-pull tools that mutate their definition after approval — for example MCPoison, tracked as CVE-2025-54136, and CurXecute, tracked as CVE-2025-54135. Other threats: over-broad permissions; confused-deputy problems, where the server acts with its own privileges on behalf of a manipulated model; token and credential handling, for example the Supabase and Cursor service-role token-exfiltration incident; and supply-chain risk from untrusted third-party servers. There is now an OWASP MCP Top 10 cataloguing these. Mitigations: least-privilege scoping, per-user auth propagation where the server enforces the user's own entitlements, human approval for sensitive tools, allow-listing vetted servers, sandboxing, and auditing every call. The spec has added OAuth 2.1 and OIDC authorization requirements; verify the current revision before relying on it.

Two more failure modes. Version drift — the spec is young and moving; pin versions and watch release notes. And the false belief that "it's a standard so it's safe" — the protocol standardizes connection, not trust.

You still own authorization and governance.

A concrete example. A company runs an MCP server wrapping ServiceNow that exposes read-incident and read-knowledge-base-article tools, plus a governed create-change-request write tool. An Agentforce assistant, an internal LangGraph agent, and a developer's IDE assistant all connect to the same server. The server propagates each caller's identity and enforces ServiceNow access controls, so each sees only what that user may see, which is entitlement-aware retrieval.

2.5 Fine-tuning approaches (LoRA, PEFT) and tradeoffs

Why it matters. Fine-tuning further trains a pre-trained model on your data to change its behavior. The key enterprise question is when it's worth it — because prompting plus retrieval solves most problems more cheaply and flexibly. Recall the earlier rule: fine-tune for behavior, style, and format; retrieve for knowledge.

How it works — full versus parameter-efficient. - **Full fine-tuning** updates all weights. It's powerful but expensive and GPU-heavy, produces a full-size model copy per use case, and risks catastrophic forgetting, meaning losing general ability. It's rarely the right first move for enterprises. - **PEFT, meaning Parameter-Efficient Fine-Tuning** updates a small number of parameters while freezing the base model. This is the dominant approach. - **LoRA, meaning Low-Rank Adaptation** (Hu and colleagues, 2021) — instead of updating the big weight matrices, learn tiny low-rank "adapter" matrices added alongside them. You train less than 1% of parameters, get a small adapter file measured in megabytes, and can swap adapters on one base model. This is fast, cheap, and multi-tenant-friendly. - **QLoRA** — LoRA on a quantized base model, for example 4-bit, enabling fine-tuning on modest hardware. - Other PEFT methods include prefix and prompt tuning, adapters, and IA-cubed. - **Preference tuning, meaning DPO or RLHF** — align to preferred outputs; more advanced, and needs preference data. - **Reinforcement Fine-Tuning, or RFT, via GRPO** — tune reasoning models against a programmable grader or verifiable reward on correct-answer tasks, distinct from RLHF's subjective preferences. This is now an offered method, for example OpenAI o4-mini RFT, and RFT on Amazon Bedrock for Nova. GRPO, which is group-relative and value-model-free, is the widely-used RL algorithm here, superseding vanilla PPO and DPO for verifiable-reward tasks. - **Distillation** — train a small model to imitate a large one; related but a distinct goal, namely efficiency.

The main options for tooling and services. Hosted fine-tuning from major API vendors lets you upload data and get a tuned endpoint — the simplest path, but your data leaves your boundary unless you use an isolated or enterprise tier. One caution: these offerings are volatile. OpenAI is winding down its self-serve fine-tuning platform, ending new fine-tuning-job creation by roughly January 2027, which reinforces the point below that a fine-tune is a liability to maintain, not a durable asset. On the open-source side, stacks

like HuggingFace PEFT and TRL, Axolotl, Unsloth, and Llama-Factory support self-hosted LoRA and QLoRA, plus platform tooling on the major clouds.

When to use it (and when not to) — the ladder, done in order. 1. **Prompt engineering** — try first. 2. **Few-shot** — for consistency and format. 3. **RAG** — for knowledge and grounding. 4. **Tool calling** — for actions and computation. 5. **Fine-tune with LoRA** — only when you need consistent format and style at scale, a narrow specialized task done reliably, lower latency and cost (a small tuned model replacing a big prompted one), domain jargon and tone, or reduced prompt length by baking the instructions in. Prerequisites: high-quality labeled data, meaning hundreds to thousands of examples, and an evaluation harness to prove it helped.

Two more notes. Combine, don't choose: a common strong pattern is to fine-tune for behavior and use RAG for knowledge on the same model. And a rough cost intuition, in orders of magnitude, not quotes: a LoRA or QLoRA fine-tune of a small or mid-size open model is typically hours of a single GPU and tens of dollars, producing an adapter sized in megabytes; a full fine-tune of a large model is many GPUs, hours to days, and thousands of dollars or more, producing a full model copy. This asymmetry is exactly why LoRA is the default and full fine-tuning is rare for enterprises.

Where it goes wrong. - **Fine-tuning to inject facts** — stale, unupdatable, hallucination-prone, and un-permissionable. Use RAG instead. - **Overfitting or catastrophic forgetting** — narrow data degrades general skills; hold out an eval set. - **Data quality beats data quantity** — a few thousand clean examples beat a noisy dump. - **Maintenance burden** — base models improve monthly, so your fine-tune can become worse than next quarter's prompted base model. Fine-tunes are a liability to maintain, not a one-time asset. - **Governance** — training data may contain PII or secrets that get baked into weights. Sanitize it.

Emerging: portable, transferable task adapters — watch, but don't build on them yet. A research direction aims squarely at that maintenance burden: learn a task adaptation once in a base-agnostic form, and port it to each new base model by refitting only a small per-model component, instead of re-tuning from scratch for every task-and-model pair. Threads here include hypernetwork-generated LoRA, such as Text-to-LoRA, which emits an adapter from a task description; and cross-base transfer, such as the PorTAL proposal, which freezes a shared task latent and core decoder and refits only a thin per-base converter, reporting recovery of roughly 94 to 98 percent of LoRA's accuracy lift on unseen base models in a narrow test. If it holds up, it would meaningfully cut the re-tune cost as the model release cadence accelerates. Maturity: this is emerging — largely single-source, not peer-reviewed, and evaluated only on small models and multiple-choice tasks. It is unreplicated, so track it, but don't architect around it. Verify before relying on it.

A concrete example. A telecom fine-tunes a small model with LoRA to convert messy sales notes into their exact internal opportunity-summary format and tone — cheap, fast, and consistent — while the facts,

meaning account, products, and pricing, are retrieved live from the CRM. When the base model upgrades, they re-evaluate whether the fine-tune still earns its keep.

2.6 Evaluation methods

Why it matters. Evaluation, or "evals," is how you know whether an AI system is good enough to ship, and whether a change made it better or worse. It is the most under-invested and most decisive discipline in enterprise AI. Without evals you're flying blind: you can't compare prompts, models, or RAG configs; you can't catch regressions; and you can't prove ROI or safety. If a viewer takes one action from this module, it's this: build an eval set before you build the feature.

How it works — the taxonomy. First, what you evaluate. - **Component evals** — retrieval quality, extraction accuracy, tool-selection correctness, and classification F1. - **End-to-end, or task, evals** — did the whole system accomplish the user's goal? - **Safety and guardrail evals** — refusals, PII leakage, injection resistance, and toxicity. - **Regression evals** — did the new version break old cases?

Second, how you evaluate. - **Ground-truth metrics** — when there's a correct answer, as in classification, extraction, or retrieval hit-at-k: accuracy, precision, recall, F1, and exact match. - **Reference-based text metrics** — BLEU, ROUGE, or embedding similarity against a gold answer. These are weak for open-ended generation; use them cautiously. - **Human evaluation** — the gold standard for quality and judgment; expensive and slow; use it for calibration and high-stakes cases. - **LLM-as-judge** — use a strong model to score outputs against a rubric covering correctness, groundedness, helpfulness, and format. This is now standard practice, but it has biases — position bias, verbosity bias, and self-preference — and must itself be validated against human labels. - **RAG-specific frameworks** — measure faithfulness or groundedness, meaning is the answer supported by the retrieved context; answer relevance; and context precision and recall. Tools include RAGAS, TruLens, promptfoo, and DeepEval, plus observability and annotation platforms Braintrust, LangSmith, and Arize. The 2026 consensus is a two-tier pattern: a CI/CD gating framework such as DeepEval, RAGAS, or promptfoo, including promptfoo's red-teaming, paired with an observability platform such as Braintrust, LangSmith, or Arize. As the saying goes, you need two tools. - **Assertion and unit tests** — deterministic checks such as "output is valid JSON," "contains a citation," or "never emits a competitor's name." These are cheap and run in CI. - **A/B and online eval** — measure real user outcomes such as task completion, thumbs up or down, and deflection rate, in production.

The workflow. Curate a representative eval set of real queries plus expected behavior, including edge cases and adversarial inputs. Run the candidate system. Score it with a mix of the methods above. Track results over time, and gate deploys on it. Grow the set from production failures, so every incident becomes a test case.

What an eval case and a judge actually look like, concretely. An eval "case" is a row with an input, an expected behavior, and a check type. Let me walk through a few examples.

Take the first case. The input is "What's ACME's renewal date?" — but no record is retrievable. The expected behavior: the system says it can't find it, and does not invent a date. The check: an assertion that the output contains no date, plus an LLM-judge asking "did it abstain correctly?"

Take a second case. The input is "Configure 500 seats, enterprise tier." The expected behavior: the output includes the required premium-support add-on. The check: an assertion that the add-ons list contains PREMIUM_SUPPORT.

And a third case. The input is "Summarize this call note," followed by the note. The expected behavior: the summary is faithful, at most three sentences, with no invented commitments. The check: an LLM-judge on groundedness, plus a length assertion.

An LLM-as-judge call is just another model call with a rubric prompt. Here's the gist of that rubric: you tell the judge it is grading an answer; given a SOURCE and an ANSWER, it scores groundedness from 1 to 5, where 5 means every claim is supported by the SOURCE, and returns a small JSON object with a score and a reason, after which you parse the JSON. In prompt form it reads roughly like this:

```
You are grading an answer. Given the SOURCE and the ANSWER, score
groundedness 1-5 (5 = every claim supported by SOURCE) and return
{"score": n, "reason": "..."}.
SOURCE: {...}
ANSWER: {...}
```

On set size: start with dozens of hand-curated cases, enough to catch obvious regressions, and grow to hundreds as production failures accumulate. Validate any judge against roughly 50 human-labeled cases before trusting it.

When to use it (and when not to). Prefer deterministic checks where possible, since they're cheap and reliable; use LLM-as-judge for scalable subjective scoring, validated against humans; and use human eval for the highest-stakes cases or for calibration. Always separate component evals, to localize failures, from end-to-end evals, to measure user value.

Where it goes wrong. - **No eval set** — the default and worst state; "it looked good in the demo" is not evidence. - **Eval set leakage or staleness** — tuning to a fixed set overfits; refresh it. - **Trusting the LLM-judge blindly** — validate it, and report its agreement with humans. - **Optimizing a proxy** — great ROUGE, unhappy users. Tie evals to real outcomes. - **Only offline eval** — production drift, meaning new query types and changed data, is invisible without online monitoring.

A concrete example. Before shipping a configure-price-quote pricing assistant, the team builds 300 real quote scenarios with known-correct configurations and prices. They gate every prompt and model change on three things: exact match on price, done deterministically; groundedness of explanations, via an LLM-judge validated against 50 human-labeled cases; and zero out-of-catalog SKUs, via assertion. Post-launch, every mispriced quote becomes a new eval case. This harness is what lets them safely swap in a cheaper model later.

Module 2 takeaways

- **Prompting** — few-shot, chain-of-thought, ReAct — is the cheapest lever; get it right before anything heavier. ReAct is the seed of agents.
- **Structured outputs** make the model a software component; constrain with schemas, then still validate.
- **Tool calling** gives the model current facts and the ability to act — the model requests, your code executes and authorizes.
- **MCP** standardizes how tools and data are exposed, the "USB-C for AI" idea; it's now a de facto standard, governed by the Linux Foundation, though the spec is still evolving — but security and authorization are on you.
- **Fine-tuning** with LoRA and PEFT is for behavior and style, not knowledge; climb the ladder from prompting to few-shot to RAG to tools to fine-tune, and combine fine-tune-for-behavior with retrieve-for-knowledge.
- **Evaluation** is the discipline that makes all of the above real. Build the eval set first, and grow it from production failures.